

Optimization Methods for Machine Learning and Engineering

Lecture 10 – Conic Programming

Julius Pfrommer



CC BY-SA 4.0

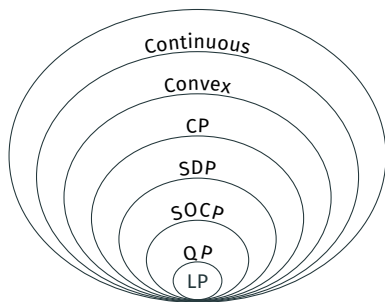
Agenda

1. Conic Programming
2. Commonly Used Cones
3. Solving Conic Problems

Conic Programming

Motivation for Conic Programming

Hierarchy of Optimization Problems



Lower levels of the hierarchy are wholly contained in the more general ones.

In this lecture we will “climb the hierarchy” from the LP upwards and see some practical examples.

Conic Programming (CP) unifies many kinds of optimization problems into a **unifying framework**.

CP allows us to **state complicated problems more compactly**.

CP lets us reason better about the **existence of a “nice” dual**.

We can find **self-concordant barriers** for quadratic convergence within the interior-point method.

Some (commercial) solvers are developed entirely around the idea of solving conic programs (MOSEK).

Different **classes of optimization problems** are **associated with a particular cone** each.

- LP: Non-Negative Orthant $\mathbb{R}_+^n$
- SOCP: Second-Order Cone $\mathcal{Q}^n$
- SDP: Semi-Definite Cone $\mathbb{S}_+^n$

Cones and Generalized Inequality

A set $\mathcal{K}$ is a **cone** if $x \in \mathcal{K} \Rightarrow \forall c \geq 0, cx \in \mathcal{K}$.

A cone $\mathcal{K}$ is a **convex cone** if

$$x_1, x_2 \in \mathcal{K} \Rightarrow \forall c_1, c_2 \geq 0, c_1 x_1 + c_2 x_2 \in \mathcal{K}.$$

A **pointed cone** is convex and does not contain straight lines passing through the origin:

$$x \in \mathcal{K} \wedge -x \in \mathcal{K} \Rightarrow x = 0$$

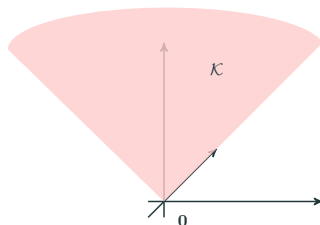
A **proper cone** is convex, pointed, closed and has a nonempty interior (beware, definitions differ between sources).

Proper cones can be used to define a **generalized inequality** with a partial ordering:

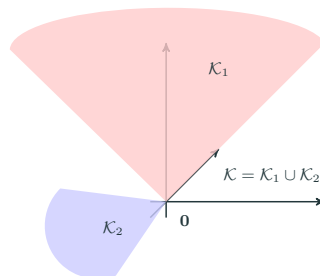
$$x \leq_{\mathcal{K}} y \Leftrightarrow y - x \in \mathcal{K}$$

$$x <_{\mathcal{K}} y \Leftrightarrow y - x \in \text{int}(\mathcal{K})$$

Convex Cone



Not-Pointed and Non-Convex Cone



Canonical Conic Program:

$$\begin{array}{ll} \min_{\mathbf{x} \in \mathbb{R}^n} & \mathbf{c}^\top \mathbf{x} \\ & \mathbf{A}\mathbf{x} - \mathbf{b} = \mathbf{0} \\ & \mathbf{x} \succeq_{\mathcal{K}} \mathbf{0} \end{array} \quad \Leftrightarrow \quad \begin{array}{ll} \min_{\mathbf{x} \in \mathcal{K}} & \mathbf{c}^\top \mathbf{x} \\ & \mathbf{A}\mathbf{x} - \mathbf{b} = \mathbf{0} \end{array}$$

Every Convex Optimization Problem is a Cone LP

$$\begin{array}{lll} \min_{\mathbf{x} \in X} & f(\mathbf{x}) & \Leftrightarrow \min_{\mathbf{y} \in \text{epi } f} y_{n+1} \Leftrightarrow \min_{\mathbf{z} \in \mathcal{X}} z_{n+1} \\ & & \text{subject to } z_{n+2} = 1 \end{array}$$

Step 1: For the general convex program, $X \subseteq \mathbb{R}^n$ is a convex set. The **epigraph of f** is again a **convex set**. Recall that $\text{epi } f = \{(\mathbf{x}, t) \mid \mathbf{x} \in X, f(\mathbf{x}) \leq t\}$.

Step 2: Construct a pointed cone $\mathcal{X}$ from the epigraph: $\mathcal{X} = \text{cl}\{(\mathbf{y}, u) \mid u > 0, (1/u)\mathbf{y} \in \text{epi } f\}$

We write $u > 0$ to avoid dividing by zero and take the **closure of the set** to include $\mathbf{0}$. Further, $\mathcal{X} \subset \mathbb{R}^{n+2}$. Constrain the solutions $\mathbf{z} \in \mathcal{X}$ to $z_{n+2} = u = 1$. Thereby, we constrain to solutions from the epigraph.

Every convex problem has an equivalent conic problem. But the cone $\mathcal{X}$ **can be very complicated** to work with. For some types of convex programs, we know **“nicer” conic equivalents**.

Commonly Used Cones

The Non-Negative Orthant

The **non-negative orthant** $\mathbb{R}_+^k$ is the cone of k -dimensional vectors with **non-negative entries**.

Every linear program can be transformed into a **conic program for the non-negative orthant**.

The inequality constraint makes use of a matrix $\mathbf{E} \in \mathbb{R}^{m \times n}$.

$$\begin{array}{lll} \min_{\mathbf{x} \in \mathbb{R}^n} & \mathbf{c}^\top \mathbf{x} & \Leftrightarrow \min_{\mathbf{x} \in \mathbb{R}^n, \mathbf{z} \in \mathbb{R}_+^m} \mathbf{c}^\top \mathbf{x} \quad \Leftrightarrow \min_{\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}_+^n, \mathbf{z} \in \mathbb{R}_+^m} \mathbf{c}^\top \mathbf{x} \\ \mathbf{Ax} - \mathbf{b} = \mathbf{0} & & \mathbf{Ax} - \mathbf{b} = \mathbf{0} \quad \mathbf{A}(\mathbf{x}_1 - \mathbf{x}_2) - \mathbf{b} = \mathbf{0} \\ \mathbf{Ex} \geq \mathbf{d} & & (\mathbf{Ex}) - \mathbf{z} = \mathbf{d} \quad (\mathbf{E}(\mathbf{x}_1 - \mathbf{x}_2)) - \mathbf{z} = \mathbf{d} \end{array}$$

Step 1: The non-negative **slack variable** $\mathbf{z}$ transforms the inequality constraint into an equality constraint.

Step 2: Replace $\mathbf{x}$ with $\mathbf{x}_1 - \mathbf{x}_2$. The new variables $\mathbf{x}_1, \mathbf{x}_2$ are both also **non-negative**.

The resulting program is **equivalent to the original LP**.

Step 3: Rewrite in **standard conic form**

$$\begin{array}{ll} \min_{\mathbf{y} \in \mathbb{R}_+^{2n+m}} & \tilde{\mathbf{c}}^\top \mathbf{y} \\ \text{s.t.} & \tilde{\mathbf{A}}\mathbf{y} - \tilde{\mathbf{b}} = \mathbf{0} \end{array}$$

where $\mathbf{y}$ corresponds to $(\mathbf{x}_1, \mathbf{x}_2, \mathbf{z})^\top$.

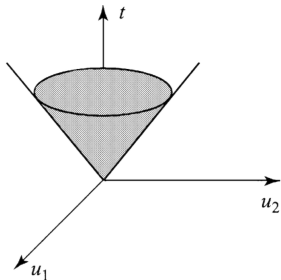
For this, $\tilde{\mathbf{c}} = (\mathbf{c}, -\mathbf{c}, \mathbf{0})^\top$ with a single affine matrix equality $\tilde{\mathbf{A}} = \begin{pmatrix} \mathbf{A} & -\mathbf{A} & \mathbf{0} \\ \mathbf{E} & -\mathbf{E} & -\mathbf{I} \end{pmatrix}$, $\tilde{\mathbf{b}} = (\mathbf{b}, -\mathbf{d})^\top$.

The Second-Order Cone

The Second-Order Cone is defined as

$$\mathcal{Q}^n = \{(\mathbf{x}, t) \in \mathbb{R}^{n-1} \times \mathbb{R} \mid \|\mathbf{x}\|_2 \leq t\}$$

The Second-Order Cone $\mathcal{Q}^3$



Many applications fall under the category of **Second-Order Cone Programming (SOCP)** [Lobo1998]. For example, for convex problems with **quadratic constraints**

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{x}^\top (\mathbf{C}^\top \mathbf{C}) \mathbf{x} + \mathbf{d}^\top \mathbf{x} + e \leq 0 \end{aligned}$$

there exists an equivalent SOCP:

$$\begin{aligned} \min_{\substack{\mathbf{x} \in \mathbb{R}^n, \\ (\mathbf{u}, t) \in \mathbb{R}^{n+2}}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & \mathbf{u} = \begin{pmatrix} \mathbf{C}\mathbf{x} \\ \frac{\mathbf{d}^\top \mathbf{x} + e + 1}{2} \end{pmatrix}, \quad t = \frac{1 + \mathbf{d}^\top \mathbf{x} - e}{2}, \\ & (\mathbf{u}, t) \geq_{\mathcal{Q}^{n+2}} \mathbf{0} \end{aligned}$$

Total Variation Denoising

Original data (gray) and
denoised estimation (black)

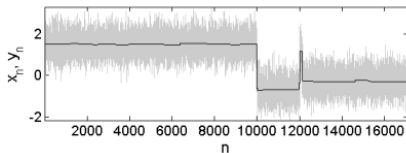


Image Source: Wikipedia

Let $\mathbf{y} \in \mathbb{R}^n$ a vector of **noisy time series** data.
The **total variation** [Rudin1992] of the time series is

$$\sum_{i=1}^{n-1} |y_{i+1} - y_i|.$$

The **denoised time series** makes a trade-off $c \geq 0$ between keeping close to the original measurements and reducing the total variation.

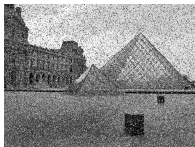
$$\min_{\mathbf{x} \in \mathbb{R}^n} \sum_{i=1}^n (y_i - x_i)^2 + c \sum_{j=1}^{n-1} |x_{j+1} - x_j|$$

The 1D denoising problem is solved with the usual trick for absolute values

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n, \mathbf{t} \in \mathbb{R}^{n-1}} \quad & \sum_{i=1}^n (y_i - x_i)^2 + c \sum_{j=1}^{n-1} t_j \\ \text{subject to} \quad & x_{j+1} - x_j < t_j, \quad j = 1, \dots, n-1 \\ & -x_{j+1} + x_j < t_j, \quad j = 1, \dots, n-1 \end{aligned}$$

Total Variation Image Denoising as SOCP [Goldfarb2005]

Gray-scale image before and after denoising



Now take a gray-scale image $\mathbf{Y} \in \mathbb{R}^{n \times m}$. The total variation is computed over all pixels (i, j) and their neighbors in two directions.

$$\|\mathbf{Y}\|_{\text{TV}} = \sum_{i,j} \sqrt{|y_{i,j+1} - y_{i,j}|^2 + |y_{i+1,j} - y_{i,j}|^2}$$

The **Frobenius Norm** of a matrix $\mathbf{X}$ is the sum of all entries individually squared $\|\mathbf{X}\|_{\text{F}} = \sqrt{\sum_{i,j} x_{ij}^2}$.

Denoising a gray-scale image is solved as an optimization problem as well:

$$\min_{\mathbf{X} \in \mathbb{R}^{n \times m}} \|\mathbf{Y} - \mathbf{X}\|_{\text{F}}^2 + c \|\mathbf{X}\|_{\text{TV}}$$

The problem of denoising can then be written with only SOCP constraints instead of the TV-norm.

$$\begin{aligned} \min_{\substack{\mathbf{X} \in \mathbb{R}^{n \times m}, \\ \mathbf{T} \in \mathbb{R}^{(n-1) \times (m-1)}}} \quad & \|\mathbf{Y} - \mathbf{X}\|_{\text{F}}^2 + c \left[\sum_{i=1}^{n-1} \sum_{j=1}^{m-1} t_{i,j} \right] \\ \text{subject to} \quad & (x_{i,j+1} - x_{i,j}, x_{i+1,j} - x_{i,j}, t_{i,j}) \in \mathcal{Q}^3, \quad \begin{aligned} i &= 1, \dots, n-1, \\ j &= 1, \dots, m-1 \end{aligned} \end{aligned}$$

Semi-Definite Programming (SDP)

Denote with $\mathbb{S}^n$ the vector space of $n \times n$ symmetric matrices with the inner product

$$\langle \mathbf{A} \mid \mathbf{B} \rangle = \text{trace}(\mathbf{A}^\top \mathbf{B}) = \sum_{i,j} a_{ij} b_{ij}.$$

- The **trace** of a square matrix is the sum of the diagonal entries.
- The trace as an inner product gives rise to the Frobenius norm.
- We could also reshape $\mathbf{A}, \mathbf{B}$ into long column vectors $\tilde{\mathbf{a}}, \tilde{\mathbf{b}}$ and write $\tilde{\mathbf{a}}^\top \tilde{\mathbf{b}}$.

We define $\mathbb{S}_+^n \subset \mathbb{S}^n$ as the cone of positive semi-definite matrices. The canonical SDP is:

$$\begin{aligned} \min_{\mathbf{X} \in \mathbb{S}_+^n} \quad & \langle \mathbf{C} \mid \mathbf{X} \rangle \\ \text{subject to} \quad & \langle \mathbf{A}_i \mid \mathbf{X} \rangle = b_i, \quad i = 1, \dots, m \end{aligned}$$

SDP are commonly used to **approximate combinatorial optimization problems**. For example:

Given a graph $G = (V, E)$, find a partition of the vertices V so as to maximize the number of edges crossing from one side to the other. The problem is NP-hard. An SDP-based relaxation finds a 0.87856-optimal solution [Goemans1995].

Wireless Sensor Localization [Luenberger2008]

Suppose you have n wireless sensors in the field. The sensor positions are $\mathbf{x}_i \in \mathbb{R}^d$ for sensors $i = 1, \dots, n$. The sensors form a mesh to communicate.

The distance between sensors $\|\mathbf{x}_i - \mathbf{x}_j\|_2^2 = d_{ij}^2$ can be estimated from the signal strength. But how to combine the different distance estimations into a joint position prediction for all sensors in the mesh?

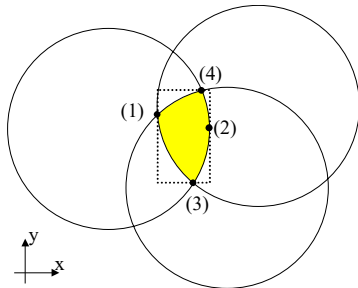
Let $\mathbf{X} = (\mathbf{x}_1 \quad \mathbf{x}_2 \quad \dots \quad \mathbf{x}_n)$ the $d \times n$ matrix of all positions.

$$\|\mathbf{x}_i - \mathbf{x}_j\|_2^2 = (\mathbf{e}_i - \mathbf{e}_j)^\top (\mathbf{X}^\top \mathbf{X}) (\mathbf{e}_i - \mathbf{e}_j)$$

where $\mathbf{e}_i$ is the vector with 1 at the i th position and zero everywhere else. Let $\mathbf{Y} = \mathbf{X}^\top \mathbf{X}$. For the joint prediction of all sensor locations, find any $\mathbf{Y}$ holding these constraints:

$$\begin{aligned} \langle (\mathbf{e}_i - \mathbf{e}_j)(\mathbf{e}_i - \mathbf{e}_j)^\top \mid \mathbf{Y} \rangle &= d_{ij}^2, \quad i, j = 1, \dots, n \\ \mathbf{Y} &\succeq_{\mathbb{S}_+^n} \mathbf{0} \end{aligned}$$

SDP-based Constraints for
Sensor Locations [Doherty2001]



Fighting Crime with Blind Deconvolutions

Suppose a **blurry image** $\mathbf{Y}$ is created from a **sharp image** $\mathbf{X}$ by the application of a (motion) **blur kernel** $\mathbf{K}$ and additional pixel-wise **random noise** $\mathbf{\Xi}$. The convolution operator is denoted as $*$.

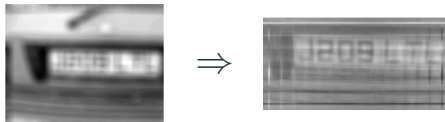


The diagram illustrates the process of creating a blurry image $\mathbf{Y}$ from a sharp image $\mathbf{X}$ and a blur kernel $\mathbf{K}$, plus noise $\mathbf{\Xi}$. It shows the equation $\mathbf{Y} = \mathbf{X} * \mathbf{K} + \mathbf{\Xi}$. The image $\mathbf{Y}$ is a blurry night scene of a city with a hot air balloon. The image $\mathbf{X}$ is the same scene but sharp. The kernel $\mathbf{K}$ is a small, dark, blurry patch. The noise $\mathbf{\Xi}$ is represented by a small, dark, noisy patch.

When the blur kernel $\mathbf{K}$ is known, a so-called **deconvolution** estimates the original image:

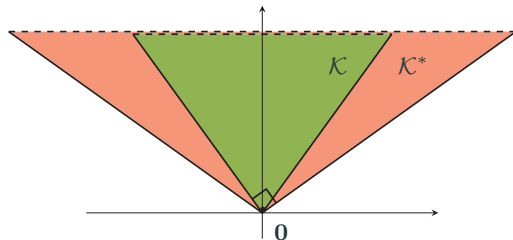
$$\min_{\hat{\mathbf{X}} \in \mathbb{R}^{n \times m}} \left[\frac{1}{2} \|\hat{\mathbf{X}} * \mathbf{K} - \mathbf{Y}\|_F^2 + c \|\hat{\mathbf{X}}\|_{TV} \right]$$

If the blur kernel is not known, a **blind deconvolution** estimates both the kernel and the original image. The **dual of this problem is an SDP** [Ahmed2013] and can be solved with “standard methods”.



Solving Conic Problems

Dual Cones



Given a cone $\mathcal{K}$, its **dual cone** is defined as

$$\mathcal{K}^* = \{s \mid \langle s \mid x \rangle \geq 0, \forall x \in \mathcal{K}\}.$$

It is easy to see that $\mathcal{K}^*$ is a cone because any $s \in \mathcal{K}^*$ can be scaled with $c \geq 0$ and the above inner product is still non-negative.

- The **dual cone of a proper cone is itself proper.**
- If a cone $\mathcal{K}$ is **closed and convex**, then $\mathcal{K} = \mathcal{K}^{**}$.
- The cones $\mathbb{R}_+^n$, $\mathcal{Q}^n$ and $\mathbb{S}_+^n$ are **self-dual**: $\mathcal{K} = \mathcal{K}^*$.

Having a closed form definition of the dual cone simplifies the following by a lot.

Primal and Dual of Conic Programs

Primal Conic Program

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} \quad & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \geq_{\mathcal{K}} \mathbf{0} \end{aligned}$$

Dual Conic Program

$$\begin{aligned} \max_{\boldsymbol{\lambda} \in \mathbb{R}^m, \mathbf{s} \in \mathbb{R}^n} \quad & \mathbf{b}^\top \boldsymbol{\lambda} \\ \text{subject to} \quad & \mathbf{A}^\top \boldsymbol{\lambda} + \mathbf{s} = \mathbf{c} \\ & \mathbf{s} \geq_{\mathcal{K}^*} \mathbf{0} \end{aligned}$$

Proof: The Lagrangian dual is $\sup_{\boldsymbol{\lambda}} \underbrace{\inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda})}_{q(\boldsymbol{\lambda})}$. A simple transformation gives:

$$\begin{aligned} q(\boldsymbol{\lambda}) &= \inf_{\mathbf{x} \in \mathcal{K}} [\mathbf{c}^\top \mathbf{x} + \boldsymbol{\lambda}^\top (\mathbf{b} - \mathbf{Ax})] = \boldsymbol{\lambda}^\top \mathbf{b} + \inf_{\mathbf{x} \in \mathcal{K}} (\mathbf{c} - \mathbf{A}^\top \boldsymbol{\lambda})^\top \mathbf{x} \\ &= \begin{cases} \boldsymbol{\lambda}^\top \mathbf{b}, & \text{if } \mathbf{c} - \mathbf{A}^\top \boldsymbol{\lambda} \in \mathcal{K}^* \\ -\infty, & \text{else} \end{cases} \end{aligned}$$

For the supremum the Lagrangian dual will not be $-\infty$ and hence $\mathbf{s} = \mathbf{c} - \mathbf{A}^\top \boldsymbol{\lambda} \in \mathcal{K}^*$.
The set of feasible solutions is closed for a proper cone $\mathcal{K}$. We can then write $\max$ instead of $\sup$. $\square$

Projection on Cones

For some cones $\mathcal{K} \subseteq \mathbb{R}^n$ the **nearest-distance projection** into the cone can be solved efficiently.

$$\mathbf{proj}_{\mathcal{K}}(\mathbf{x}) = \arg \min_{\mathbf{y} \in \mathcal{K}} \|\mathbf{x} - \mathbf{y}\|_2$$

Positive Orthant

$$\mathbf{proj}_{\mathbb{R}_+^n}(\mathbf{x}) = (\max(0, x_1), \dots, \max(0, x_n))^\top$$

Second-Order Cone

Let $\mathbf{x} = (\mathbf{x}_-, t)$, $\kappa_i(\mathbf{x}) = t + (-1)^i \|\mathbf{x}_-\|_2$ and $c_i(\mathbf{x}) = \begin{cases} \frac{1}{2} \left((-1)^i \frac{\mathbf{x}_-}{\|\mathbf{x}_-\|}, 1 \right)^\top, & \text{if } \mathbf{x}_- \neq \mathbf{0} \\ \frac{1}{2} \left((-1)^i \mathbf{w}, 1 \right)^\top, & \text{else} \end{cases}$ for

$\mathbf{w} \in \mathbb{R}^{n-1}$ any vector with $\|\mathbf{w}\|_2 = 1$. The projection is then a closed-form equation [Oustry2008].

$$\mathbf{proj}_{\mathcal{Q}^n}(\mathbf{x}) = \max(\kappa_1(\mathbf{x}), 0) c_1(\mathbf{x}) + \max(\kappa_2(\mathbf{x}), 0) c_2(\mathbf{x})$$

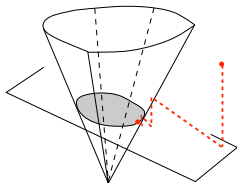
Semi-Definite Cone

Let the matrix $\mathbf{X}$ have an eigenvalue decomposition $\mathbf{X} = \mathbf{U}^\top \mathbf{diag}(\boldsymbol{\lambda}) \mathbf{U}$. The projection into the Semi-Definite Cone comes from “filtering” only the positive eigenvalues [Schwartzman1979].

$$\mathbf{proj}_{\mathbb{S}_+^n}(\mathbf{X}) = \mathbf{U}^\top \mathbf{diag}((\max(0, \lambda_1), \dots, \max(0, \lambda_n))^\top) \mathbf{U}$$

From Alternating Projections to Alternating Directions

Let $P = \{x \mid Ax = b\}$ feasible solutions for affine equality constraints and x_0 a non-feasible initial point. Find the projection on the intersection of a cone $\mathcal{K}$ and P by **alternating projections** [Dykstra1983]:



Alternating Projection with Dykstra's Correction

- 1: $z_0 \leftarrow x_0$
- 2: **while** $k = 1, \dots$ **do**
- 3: $x_{k+1} \leftarrow \text{proj}_P(z_k)$
- 4: $y_{k+1} \leftarrow \text{proj}_{\mathcal{K}}(x_{k+1})$
- 5: $z_{k+1} \leftarrow z_k - (x_{k+1} - y_{k+1})$
- 6: **end while**

Now we want to optimize over the intersection of $\mathcal{K}$ and P . **Split into two “equal” variables** x and y :
This becomes the optimization problem

$$\begin{aligned} \min_{x \in \mathcal{K}, y \in P} \quad & f(x) + g(y) \\ \text{subject to} \quad & x = y \end{aligned}$$

with the Augmented Lagrangian (for the hyperparameter ρ):

$$\mathcal{L}(x, y, \lambda) = f(x) + g(y) + \lambda^\top (x - y) + \rho \|x - y\|_2^2$$

Alternating Lagrangian Optimization

- 1: $y_0 \leftarrow x_0, \quad \lambda_0 \leftarrow 0$
- 2: **while** $k = 0, \dots$ **do**
- 3: $x_{k+1} \leftarrow \min_{x \in P} \mathcal{L}(x, y_k, \lambda_k)$
- 4: $y_{k+1} \leftarrow \min_{y \in \mathcal{K}} \mathcal{L}(x_{k+1}, y, \lambda_k)$
- 5: $\lambda_{k+1} \leftarrow \lambda_k - \rho(x_{k+1} - y_{k+1})$
- 6: **end while**

See [Henrion2012] and more general [Boyd2011]

Barriers for using the Interior Point Method for Conic Programs

In Lecture 3 we used the logarithmic barrier for individual inequality constraints $g_i(\mathbf{x}) \leq 0$. For the standard cones, similar barrier functions are available:

Cone	Barrier Function
Non-Negative Orthant $\mathbf{x} \in \mathbb{R}_+^n$	$\phi(\mathbf{x}) = -\sum_{i=1}^n \log(x_i)$
Second-Order Cone $(\mathbf{u}, t) \in \mathcal{Q}^n$	$\phi(\mathbf{u}, t) = -\log(t^2 - \ \mathbf{u}\ _2^2)$
Semi-Definite Cone $\mathbf{X} \in \mathbb{S}_+^n$	$\phi(\mathbf{X}) = -\log(\mathbf{X})$

The barriers can be used with the **Interior Point Method to solve conic problems**. Possibly as a “subroutine” for the alternating directions method from the previous slide.

Note the deep theory of “self-concordant barriers” [Nesterov1994] to optimally increase the scaling factor for tightening the barrier (beyond our scope).

The barrier for the Semi-Definite Cone is motivated as follows. A symmetric real $n \times n$ matrix $\mathbf{X}$ is **positive semi-definite iff all its eigenvalues are non-negative**. The determinant $|\mathbf{X}|$ is the product of its n eigenvalues.

$$\sum_{i=1}^n \log(\text{eig}_i(\mathbf{X})) = \log\left(\prod_{i=1}^n \text{eig}_i(\mathbf{X})\right) = \log(|\mathbf{X}|)$$

Summary of what you learned today

- **Convex cones** giving rise to classes of optimization problems
- Common “nice” cones with applications
 - The **Positive Orthant**
 - The **Second-Order Cone**
 - The **Semi-Definite Cone**
- Important concepts for solving conic problems with numerical algorithms
 - **Dual Cones** and the **Lagrangian dual of conic problems**
 - **Projection** into a cone
 - **Alternating methods** where separated subproblems are solved more efficiently
 - **Barrier functions** for cone inequalities to be used in Interior Point Methods

- [Ahmed2013] Ali Ahmed, Benjamin Recht, and Justin Romberg. “Blind deconvolution using convex programming”. In: *IEEE Transactions on Information Theory* 60.3 (2013), pp. 1711–1732.
- [Andersen2003] Erling D Andersen, Cornelis Roos, and Tamas Terlaky. “On implementing a primal-dual interior-point method for conic quadratic optimization”. In: *Mathematical Programming* 95.2 (2003), pp. 249–277.
- [Boyd2011] Stephen Boyd, Neal Parikh, and Eric Chu. *Distributed optimization and statistical learning via the alternating direction method of multipliers*. Now Publishers Inc, 2011.
- [Doherty2001] Lance Doherty, Laurent El Ghaoui, et al. “Convex position estimation in wireless sensor networks”. In: *Proceedings IEEE INFOCOM 2001. Conference on Computer Communications. Twentieth Annual Joint Conference of the IEEE Computer and Communications Society (Cat. No. 01CH37213)*. Vol. 3. IEEE. 2001, pp. 1655–1663.
- [Dykstra1983] Richard L Dykstra. “An algorithm for restricted least squares regression”. In: *Journal of the American Statistical Association* 78.384 (1983), pp. 837–842.
- [Goemans1995] Michel X Goemans and David P Williamson. “Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming”. In: *Journal of the ACM (JACM)* 42.6 (1995), pp. 1115–1145.

- [Goldfarb2005] Donald Goldfarb and Wotao Yin. “Second-order cone programming methods for total variation-based image restoration”. In: *SIAM Journal on Scientific Computing* 27.2 (2005), pp. 622–645.
- [Henrion2012] Didier Henrion and Jérôme Malick. “Projection methods in conic optimization”. In: *Handbook on Semidefinite, Conic and Polynomial Optimization* (2012), pp. 565–600.
- [Lai2016] Wei-Sheng Lai et al. “A comparative study for single image blind deblurring”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2016, pp. 1701–1709.
- [Lobo1998] Miguel Sousa Lobo et al. “Applications of second-order cone programming”. In: *Linear algebra and its applications* 284.1-3 (1998), pp. 193–228.
- [Nesterov1994] Yurii Nesterov and Arkadii Nemirovskii. *Interior-point polynomial algorithms in convex programming*. SIAM, 1994.
- [Nesterov1998] Yu E Nesterov and Michael J Todd. “Primal-dual interior-point methods for self-scaled cones”. In: *SIAM Journal on optimization* 8.2 (1998), pp. 324–364.
- [Outrata2008] Jiří V Outrata and Defeng Sun. “On the coderivative of the projection operator onto the second-order cone”. In: *Set-Valued Analysis* 16.7 (2008), pp. 999–1014.

- [Rudin1992] Leonid I Rudin, Stanley Osher, and Emad Fatemi. “Nonlinear total variation based noise removal algorithms”. In: *Physica D: nonlinear phenomena* 60.1-4 (1992), pp. 259–268.
- [Schwertman1979] Neil C Schwertman and David M Allen. “Smoothing an indefinite variance-covariance matrix”. In: *Journal of Statistical Computation and Simulation* 9.3 (1979), pp. 183–194.